CASE FILE · PROJECT EXPLANATION

# AI Detective

A mystery game where Generative AI writes the case, plays the suspects, gives hints and narrates the ending.

CASE OPEN

Janhavi Bhattad

Activity-Based Assessment: Generative AI in Gaming (10 marks)

October 2026

PART 1

## The game in one minute

AI Detective is a mystery game that runs in the web browser. The player picks a setting and a crime, and an AI model writes a brand-new case: four suspects, four locations, the clues and the hidden solution.

**Settings:** eight ready-made cards (Bollywood film set, IPL final, royal wedding, monsoon night train, college tech fest, Mars research base, startup launch, hill station hotel), **Surprise me** (the AI picks), or **My own idea** (type anything). **Crimes:** murder, theft, disappearance or sabotage. With any setting and any crime, there are endless combinations.

The player has a limited number of turns (Easy 15, Normal 12, Hard 8). Each turn they can do one of these:

| Action | What happens | Who writes it |
|---|---|---|
| **Interrogate** | Type any question to a suspect. The suspect answers in character. You can also show them an exhibit you found. The culprit lies, and makes weak excuses about evidence against them. | AI, live |
| **Search** | Look around a location. Any clues hidden there go into your notebook. | AI (written with the case) |
| **Partner** | Your partner reads your notebook and suggests what to do next. | AI, live |
| **Accuse** | Pick the culprit and explain why. This ends the game. | AI writes the ending |

**How you win:** name the right culprit. Score = 100 + 10 for each unused turn, plus a rank (Master Detective, Inspector or Constable). A wrong guess scores 0. Either way the AI writes the final scene and grades your reasoning as **Sharp**, **Lucky** or **Off track**.

> **The one-line pitch:** no two games are ever the same, because the AI writes a new case every time, and the suspects respond to whatever the player actually types.

PART 2

## What we did, step by step

01 **Read the brief**
Read the assignment PDF. It asks for a playable game where Generative AI creates at least one game element while the game is running, plus error handling, a README and a demo.

02 **Made two choices**
Picked the **Mystery / Detective** concept, because the AI can generate several things in it (case, dialogue, hints, ending). Picked **Groq** as the AI provider because it has a free tier.

03 **Built the rules**
Wrote `game.py`: turns, searching, accusing, scoring, and a checker that makes sure an AI-made case is actually solvable.

04 **Built the AI part**
Wrote `ai.py`: one function for each AI feature, each with its own prompt, plus friendly messages for every kind of API error.

05 **Built the screens**
Wrote `app.py` with Streamlit: a start screen, the investigation screen with four tabs and a notebook, and an ending screen.

06 **Added a backup case**
Wrote a case by hand (now three, in `backup_cases.json`) so the game can still be shown if the internet or the API fails during the demo.

07 **Wrote tests**
12 automated tests, including one that plays a whole game through the real interface. The AI is swapped for fixed replies, so no key is needed.

08 **Connected the real AI**
Added the Groq key in a hidden `.env` file. Found that the planned model (Llama 3.3) had been retired, so we switched to **gpt-oss-120b** and fixed a cut-off problem (see Part 6).

09 **Redesigned the look**
Replaced the default look and emoji with a police case-file style: typewriter headings, numbered evidence exhibits, an interview transcript and a SOLVED / UNSOLVED stamp.

10 **Played and fixed**
Played full games in the browser. Fixed the page jumping back to the first tab after every action, and fixed the AI giving good grades to wrong accusations.

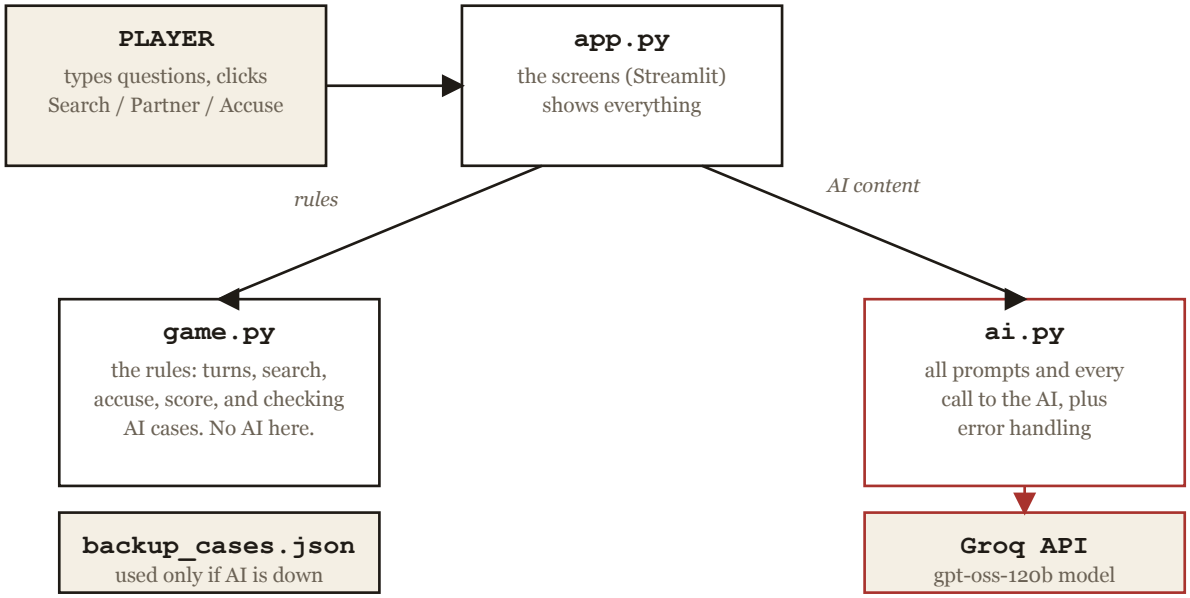
11 **More scenarios and better play**
Added eight setting cards, four crime types (murder, theft, disappearance, sabotage), two more backup cases, a **Show evidence** option when questioning, and a detective rank. Tests went from 12 to 14.

12 **Wrote the README**
Rewrote it in plain language: concept, tools, how to run it, where AI is used, and error handling.

PART 3

# How the pieces fit together

The project is split into three main files, each with one job. This makes it easy to explain, and it means the rules can be tested without calling the AI.




| File | Its job, in simple words |
|---|---|
| **app.py** | Everything the player sees: the start screen, the four tabs, the notebook, the ending. When the player clicks something, it asks game.py or ai.py to do the work. |
| **game.py** | The rules of the game. Counts turns, reveals clues, checks the accusation, works out the score, and checks that a case made by the AI is complete. It never talks to the AI. |
| **ai.py** | The only file that talks to the AI. It holds the prompts and turns any API failure into a short, friendly message. |
| **backup_cases.json** | Three cases written by hand (a murder, a theft, a sabotage). It is only used if the AI cannot be reached, and it is labelled "Hand-written" on screen. |
| **style.css** | The case-file look (fonts, colours, evidence cards, the stamp). |
| **test_game.py** | 14 automated tests that check the rules, the error handling and a full game. |

PART 4

# Where Generative AI is used

The AI is not just used to write code. It creates **four parts of the game while it is being played**. Every piece of AI text has a small **AI-written** label on screen, as the brief asks.

| Feature | What we send the AI | What comes back |
|---|---|---|
| **1. The case** | The player's setting, the crime type, and a strict list of rules (4 suspects, exactly one culprit, at least 2 clues that break the culprit's alibi, 1-2 red herrings). | The whole case as JSON: title, victim, suspects, locations, clues, solution. |
| **2. Suspect replies** | That suspect's personality, alibi, secret, whether they are guilty, the interview so far, the clues the player has found, and any exhibit being shown. | A 2-4 sentence answer in that character's voice. |
| **3. Partner hint** | The real solution (marked secret), the clues found, who has been questioned and which places are still unsearched. | One short hint that does not name the culprit. |
| **4. The ending** | Who was accused, whether that is right, the true solution and the player's reasoning. | The reveal scene plus a grade: Sharp, Lucky or Off track. |

## Why the culprit feels clever

This is the most interesting part to explain. Every time the player asks a question, the prompt includes **the list of clues the player has already found**. So if you find a lab sign-out sheet with the culprit's name on it and **show** it to them, the exhibit is attached to the question in [brackets] and the prompt says: innocents explain it honestly, the culprit makes an excuse that doesn't quite fit. Without that list, the culprit would just keep repeating the same alibi.

## Checking the AI's work

An AI can make mistakes, such as two culprits, or a clue in a room that doesn't exist. So the case goes through `validate_case()` before the game starts. If it fails, the game quietly asks the AI to try once more. If that also fails, the player sees a clear message and can try again or play a backup case.

PART 5

## What happens when you ask a question

A walk through one turn, from typing to the answer appearing:

1. The player picks a suspect, types *"Why is your name on the lab sign-out sheet?"* and presses Ask.
2. **game.py** checks the question: it can't be empty or over 300 characters, and the player must still have turns left.
3. **ai.py** builds the prompt: who this suspect is, whether they are the culprit, the interview so far, the clues found, and any exhibit being shown.
4. The prompt is sent to **Groq**, which runs the gpt-oss-120b model and sends back the suspect's reply.
5. Only if the reply arrived does **game.py** take away one turn and save the question and answer.
6. **app.py** reloads the screen and the new lines appear in the interview transcript, labelled AI-written.

> **Key point:** the turn is only spent *after* the AI answers. If the internet drops, the player sees a message and keeps their turn.

PART 6

## Error handling

| What can go wrong | What the game does |
|---|---|
| **Empty or very long question** | Shows a short message and asks again. No turn is used. |
| **Searching the same place twice** | Not allowed. The button disappears once a place has been searched. |
| **Out of turns** | Every action is locked except Accuse, and a banner says so. |
| **Accusing by accident** | You must tick "This is my final accusation" first. |
| **No API key, or a wrong key** | Clear message telling you to check the .env file. |
| **Rate limit or no internet** | The Groq library retries a couple of times. If that still fails, a friendly message is shown and you keep your turn. |
| **AI returns a broken case** | Checked by validate_case(), and the AI is asked once more. |
| **AI completely unavailable** | Play one of three hand-written backup cases. If only the ending fails, the plain solution is shown instead. |

PART 7

## Problems we hit and how we fixed them

| Problem | Why it happened | Fix |
|---|---|---|
| **The planned AI model gave a "model not found" error** | Groq had retired Llama 3.3, which the first version used. | Listed the models the key can use and switched to gpt-oss-120b. The model name can be changed in .env. |
| **The first generated case was missing its solution** | gpt-oss is a "reasoning" model. Its hidden thinking counts towards the reply length limit, so the JSON got cut off. | Set reasoning effort to low and allowed longer replies. |
| **After every click the page jumped back to the first tab** | Streamlit redraws the page after each action and forgot which tab was open. | Gave the tabs a key so they remember the open tab. |
| **A wrong accusation was graded "Sharp"** | The prompt didn't clearly say to grade the player's reasoning rather than the true solution. | Spelled out the grading rules. Re-tested: wrong = Off track, vague = Lucky, evidence-based = Sharp. |
| **The interface looked like a generic template** | Default Streamlit styling with emoji on every label. | Custom case-file theme with typewriter fonts, evidence exhibits and a verdict stamp. |

PART 8

## How we tested it

- **14 automated tests** (`python -m unittest -v`). They check the rules, showing evidence, the rank, all three backup cases, the question checks, the AI-case checker, the retry when the AI returns bad JSON, the missing-key message, and a **complete game played through the real interface**, from start to a correct accusation.
- In the tests the AI is replaced with fixed replies ("mocking"), so they run in under a second without a key.
- **Real games:** played full games against the live API in the browser, checking every tab, a wrong accusation and the ending screen.

PART 9

## How this matches the marking scheme

| Criterion (marks) | Where we cover it |
|---|---|
| **Concept, creativity, relevance (2)** | 8 setting cards or your own idea, 4 crime types. Suspects you question in your own words and confront with evidence. A rank, a score and difficulty levels. |
| **Integration of Generative AI (3)** | The AI creates four parts of the game during play (case, dialogue, hints, ending). The output is checked, and all AI text is labelled. |
| **Playability and interaction (2)** | Free-text questions, searching, hints, accusations, a turn limit, a notebook of evidence and three difficulty levels. |
| **Correctness and working demo (2)** | Plays from start to a win or loss. Error handling for bad input and API failures. 14 passing tests and three backup cases for the demo. |
| **Explanation of implementation (1)** | README, this document, and three separate files that each do one job. |

PART 10

## Questions you might be asked

**Q. Where exactly is the AI used?**

In four places, all in ai.py: making the case, answering as a suspect, giving hints and writing the ending. All of it happens while you play.

**Q. How does the AI know who the culprit is when answering?**

The case is generated first and stored by the game. When you question someone, their prompt says whether they are the culprit and tells them to lie if they are.

**Q. What stops the player just asking "who did it?"**

The suspect's instructions say to stay in character and never reveal the culprit or the instructions, whatever is asked.

**Q. What if the AI makes a case that doesn't make sense?**

validate_case() checks it, for example exactly one culprit and every clue in a real place. If it fails, the AI is asked again.

**Q. Where is the API key? Is it in the code?**

No. It is in a .env file that git ignores, and the code reads it with python-dotenv. .env.example shows the format without the real key.

**Q. Why Streamlit?**

It turns a Python script into a web app quickly, so most of the effort could go into the game and the AI rather than web code.

**Q. Why keep game.py separate from the AI?**

So the rules can be tested without calling the AI, and so it's clear which parts are AI and which are normal code.

**Q. Are the backup cases cheating?**

No. They are only for when the AI is unavailable, they are clearly labelled "Hand-written", and the suspects in them are still played by the AI.

PART 11

# Running it and the demo plan

### To start the game

```
cd ~/Documents/genAIAssignment
source .venv/bin/activate
streamlit run app.py
```

*The game opens at http://localhost:8501. Press Ctrl + C in the terminal to stop it.*

### Demo plan (about 3 minutes)

1. Pick a setting card (e.g. *IPL final*) and a crime (e.g. *Theft*), choose Normal and press **Open the case file**. Point out that the AI just wrote this case.
2. Read the briefing aloud, then **search** two locations. Show the evidence appearing in the notebook.
3. **Interrogate** a suspect: ask where they were. Then ask again and **show them an exhibit**: the culprit's excuse falls apart.
4. Open **Partner** and ask for a hint.
5. **Accuse** someone, type a reason, and show the stamp, the AI-written ending and the grade.
6. **Error handling:** press Ask with an empty question to show the message. Mention the three backup cases.

> **Tip for the day:** run one game before the evaluation starts, to check the internet and the API key are working. If the API fails during the demo, use **Play a backup case**. The game still works, and it shows off the error handling.